



# AI SOFTWARE DEVELOPMENT INTERNSHIP

EMAN MASOOD (BSSE)

Week 5 — Detailed Task Sheet

Embeddings & Vector Databases • RAG Fundamentals Manual Retrieval-  
Augmented Generation Pipeline • Git — `git revert`

## PART A — EMBEDDINGS FUNDAMENTALS

### Practice Tasks

1. Run the script above and confirm the "similar meaning" score is noticeably higher than the "unrelated meaning" score.

### Outputs:

```
Problems Output Debug Console Terminal Ports
(venv) PS D:\Week5\Week5_PartA_Embeddings> pip install openai
Using cached annotated_types-0.8.0-py3-none-any.whl.metadata (4.5 kB)
Collecting pydantic-core==2.46.5 (from pydantic!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.*,<3,>=1.10.13->openai)
Using cached pydantic_core-2.46.5-cp313-cp313-win_amd64.whl.metadata (6.7 kB)
Collecting typing-inspection>=0.4.2 (from pydantic!=2.0.*,!=2.1.*,!=2.2.*,!=2.3.*,<3,>=1.10.13->openai)
Using cached typing_inspection-0.4.4-py3-none-any.whl.metadata (2.6 kB)
Downloading openai-3.14.1-py3-none-any.whl (2.0 MB)
2.0/2.0 MB 72.5 kB/s 0:00:26
Using cached anyio-4.15.1-py3-none-any.whl (132 kB)
Downloading httpx2-2.13.0-py3-none-any.whl (95 kB)
Downloading httpcore2-2.13.0-py3-none-any.whl (83 kB)
Downloading jiter-0.17.0-cp313-cp313-win_amd64.whl (226 kB)
Using cached pydantic-2.13.5-py3-none-any.whl (472 kB)
Using cached pydantic_core-2.46.5-cp313-cp313-win_amd64.whl (2.0 MB)
Using cached typing_extensions-4.16.0-py3-none-any.whl (45 kB)
Using cached annotated_types-0.8.0-py3-none-any.whl (13 kB)
Using cached h11-0.16.0-py3-none-any.whl (37 kB)
Using cached idna-3.19-py3-none-any.whl (68 kB)
Using cached truststore-0.10.4-py3-none-any.whl (18 kB)
Using cached typing_inspection-0.4.4-py3-none-any.whl (14 kB)
Using cached sniffio-1.3.1-py3-none-any.whl (10 kB)
Installing collected packages: typing-extensions, truststore, sniffio, jiter, idna, h11, annotated-types, typing-inspection, pydantic-core, httpcore2, anyio, pydantic, httpx2, openai
Successfully installed annotated-types-0.8.0 anyio-4.15.1 h11-0.16.0 httpcore2-2.13.0 httpx2-2.13.0 idna-3.19 jiter-0.17.0 openai-3.14.1 pydantic-2.13.5 pydantic-core-2.46.5 sniffio-1.3.1 truststore-0.10.4 typing-extensions-4.16.0 typing-inspection-0.4.4
[notice] A new release of pip is available: 26.1.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip

● (venv) PS D:\Week5\Week5_PartA_Embeddings> python embed_demo.py
similar meaning: 0.8228791896025641
unrelated meaning: 0.5355357921114333
○ (venv) PS D:\Week5\Week5_PartA_Embeddings> |
```

2. Print `len(v1)` and confirm the embedding is a long list of plain numbers — there is nothing readable in it by itself; the meaning only shows up when comparing vectors.

Output:

```
(venv) PS D:\Week5\Week5_PartA_Embeddings> python embed_demo.py
similar meaning: 0.8228791896025641
unrelated meaning: 0.5355357921114333
embedding length: 3072
(venv) PS D:\Week5\Week5_PartA_Embeddings> ^C
(venv) PS D:\Week5\Week5_PartA_Embeddings>
```

3. Try two sentences that share many of the same words but mean opposite things (e.g. "I loved this book" vs "I did not love this book") and see how close or far apart they land — write down what you observe.

Output:

```
(venv) PS D:\Week5\Week5_PartA_Embeddings> ^C
(venv) PS D:\Week5\Week5_PartA_Embeddings> python embed_demo.py
similar meaning: 0.8228791896025641
unrelated meaning: 0.5355357921114333
embedding length: 3072
opposite meaning: 0.7260658159270806
(venv) PS D:\Week5\Week5_PartA_Embeddings>
```

**Observation:** The two sentences have many of the same words, so their embeddings are still fairly close (cosine similarity  $\approx 0.726$ ). This shows that embeddings capture overall semantic similarity, but they may not fully distinguish the meaning change caused by negation.